

DPP Assignment 3

Simon Vinding Brodersen(rzn875)

December 9, 2025

1 Tree Operations

1.1 Implement *depths*

Below my implementation of the depths can be seen, which seems to work.

```
def depths (steps: []step) : [](i64, i32) =  
  let (arr, values, keep) = unzip3 <| map (\s ->  
    match s  
      case #u -> (-1, -1, false)  
      case #d x -> (1, x, true)  
  ) steps  
  let inc_scan_arr = scan (+) 0 arr  
  let exc_scan_arr = map2 (\a b -> a - b) inc_scan_arr arr  
  let (depth, value, _) = zip3 exc_scan_arr values keep |> filter (\(_, _, b) -> b) |>  
    unzip3  
  in zip depth value
```

Here we have to convert the steps into items of 1 for down and -1 for up operations. Then we can do an exclusive scan and at last remove all the elements which originally corresponded to up items as these do not add new elements to the graph.

If we let N denote the length of steps, then we have a map, a scan, another map and at last a filter, which all work on an array of the full length N . The filter and the scan operation is what is the limiting factor giving us work of $O(N)$ and span of $O(\lg N)$.

Any sequential computation would have to touch all elements at least once, and as such this implementation is work efficient.

1.2 Implement *parents*

Below is my implementation of the parents function.

```

def parents (D: []i64) : []i64 =
  let n = length D
  let depth_tuple = zip (D :> [n]i64) (iota n)
  let parents = loop parents = [0] for i' < n - 1 do
    let i = i' + 1
    let search_d = D[i] - 1
    let search_ds = map (\i -> depth_tuple[i]) (iota i)
    let (_, p) = reduce_comm (\(d1, i1) (d2, i2) ->
      if (d1 == d2) && (d1 == search_d) then
        if i1 > i2 then (d1, i1)
        else (d2, i2)
      else if d1 == search_d then (d1, i1)
      else if d2 == search_d then (d2, i2)
      else (-1, -1)
    ) (0, -1) search_ds
    in parents ++ [p]
  in parents

```

We initialize the parents with the root being itself, and then otherwise we search to the left of us for items with a value less than the current index, where we choose the higher index in case of a tie.

The algorithm runs a sequential loop over all elements (except the root). Inside this loop it runs a map and a `reduce_comm`. The `reduce_comm` only runs on the left items, meaning that the length that is searched is a reduced subset of the full list. However, as an upper bound this would still be over the full array. As such inside the loop we do $O(N)$ work with a span of $O(\lg N)$. We do this within a loop giving total work of $O(N^2)$ and span $O(N \lg N)$.

We are able to compute the parent vector by a single traversal of the original tree and as such I would not state that this algorithm is work efficient.

1.3 Implement *subtree_sizes*

Below my implementation of the *subtree_sizes* can be seen.

```

def subtree_sizes [n] (steps: [n]step) : []i64 =
  let (D, V) = depths steps |> unzip
  let P = parents D
  let n = length D
  let max_depth = reduce i64.max 0 D
  let (res, _) = loop (res, cur_depth) = (copy V, max_depth) while cur_depth > 0 do
    let indices = filter (\i -> D[i] == cur_depth) (iota n)
    let values = map (\i -> res[i]) indices
    let parent_indices = map (\i -> P[i]) indices
    in (reduce_by_index res (+) (0) parent_indices values, cur_depth - 1)
  in map i64.i32 res

```

This approach uses the naive bottom up iteration. Here we have a call to the reduce function, followed by a sequential loop, which runs over the depth of the tree. This loop calls a filter, map and `reduce_by_index`.